

University
Politehnica
of Bucharest





PL/SQL (V)

Alexandru Petrescu
apetrescu0506@stud.acs.upb.ro



Overview

- Subprograms
- Functions
- Stored Procedures
- Packages

Subprograms

Why Subprograms?

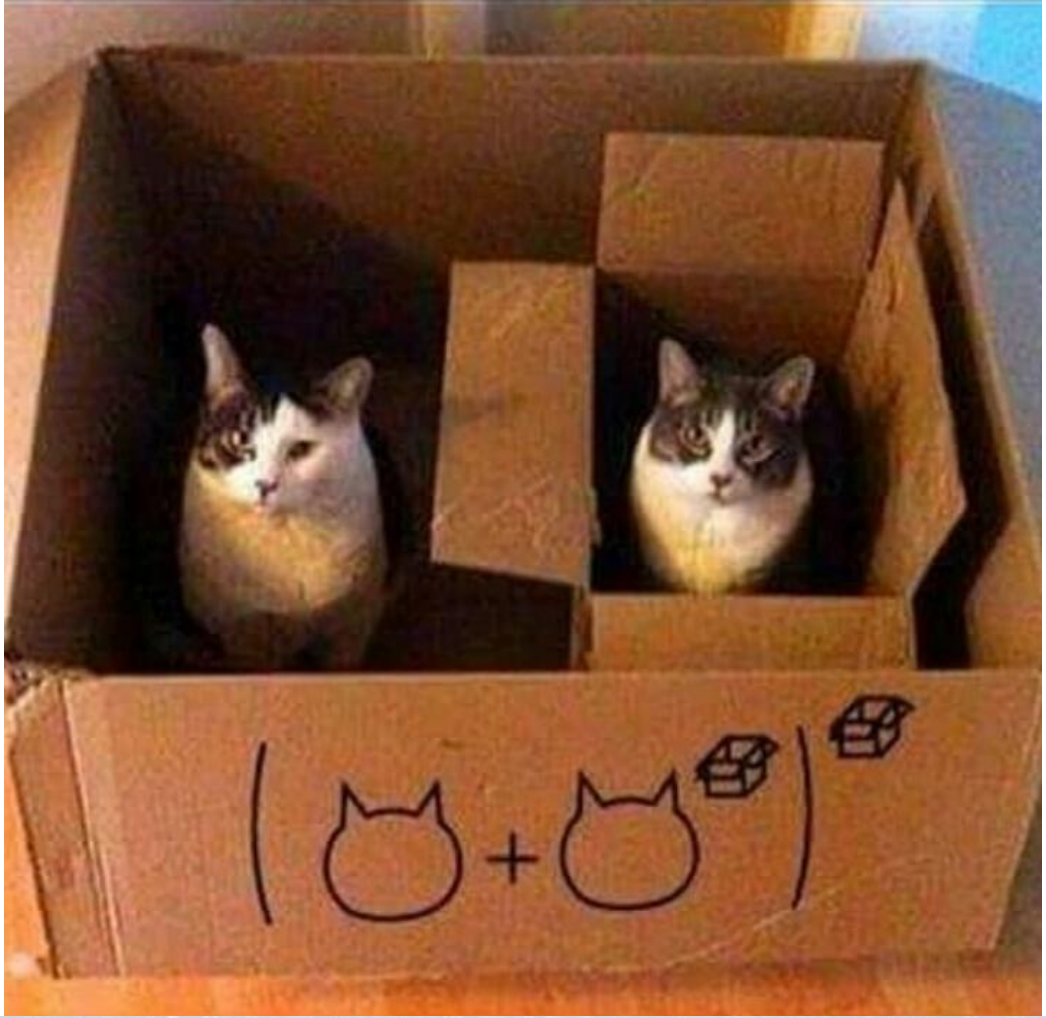
- **Modularity** : Divide and Conquer
- **Easier** Application **Design** : Can test each part
- **Maintainability** : Change just what you need
- **Packageability** : you can pack them together
- **Reusability** : Bye Bye Copy Paste
- Better **Performance**: Reduces overhead/traffic

Types of Subprograms

- **Nested**
- **Package**
- **Standalone** Subprograms

Nested

- Is a type of subprogram created **inside** a PL/SQL block
- It ca be declared at the beginning (with a header) or later
- It is **stored** in the database **only** if it is **nested in** a **standalone** or **package** subprogram


|🐱+🐱|²

Package

- It is a subprogram created **inside a package**
- You **declare** it in the **package specification** and **define** it in the **package body**
- It is **stored** in the database until you **drop** the **package**

Standalone

- It is a subprogram created at **schema level**

- You **create** it with the "*CREATE FUNCTION*" or "*CREATE PROCEDURE*" statement

- It is **stored** in the database **until** you **drop** it with the "*DROP FUNCTION*" or "*DROP PROCEDURE*" statement

Stored Subprograms

- Are either **package** or **standalone**

- Are affected by the "AUTHID" and "*ACCESSIBLE BY*" clauses, which can appear in the "*CREATE FUNCTION*", "*CREATE PROCEDURE*", and "*CREATE PACKAGE*"

Properties

- The "*AUTHID*" clause affects the name **resolution** and **privilege** checking of SQL statements that the subprogram issues at **run time**

- The "*ACCESSIBLE BY*" clause specifies a white list of PL/SQL units that can access the subprogram

Properties

- **Each** subprogram **property** can appear **only once** in the subprogram **declaration**
- The properties can appear in **any order**
- Properties appear **before** the "*IS*" or "*AS*" keyword in the subprogram **heading**
- The properties **cannot appear** in **nested** subprograms.

Properties

- **Only** the "*ACCESSIBLE BY*" property can appear in **package** subprograms. Standalone subprograms may have the following properties in their declaration.
  - "*ACCESSIBLE BY*" Clause
  - "*DEFAULT COLLATION*" Clause
  - Invoker's Rights and Definer's Rights (AUTHID Property)

Declaration

- A subprogram **begins** with a **heading**, which specifies its **name** and (**optionally**) its **parameter** list.

- Like an anonymous block, a subprogram has these parts:
  - **Declarative** part (optional)
  - **Executable** part (required)
  - **Exception-handling** part (optional)

Subprogram Invocations

```
subprogram_name [ ( [ parameter [, parameter]... ] ) ]
```

- If the subprogram has **no parameters**, or specifies a **default value** for every parameter, you can either **omit** the parameter list or specify an **empty** parameter **list**
- A procedure invocation is a PL/SQL statement
- A function invocation is an expression

Return Statement

- It immediately **ends** the **execution** of the subprogram or anonymous block that contains it

- A subprogram or anonymous block can contain **multiple** "*RETURN*" statements

- It is **mandatory** for **functions**

Return Statement

- In a procedure, the RETURN statement **returns control** to the **invoker**, where execution **resumes** immediately **after** the **invocation**

- The "*RETURN*" statement **cannot** specify an **expression**

```
1 DECLARE
2   PROCEDURE p IS
3   BEGIN
4     DBMS_OUTPUT.PUT_LINE('Inside p');
5     RETURN;
6     DBMS_OUTPUT.PUT_LINE('Unreachable statement.');
7   END;
8 BEGIN
9   p;
10  DBMS_OUTPUT.PUT_LINE('Control returns here.');
11 END;
```

Functions

A Function Has

- **Same structure** as a **subprogram**, **except** that:
  - A function **heading** must **include** a "*RETURN*" clause, which specifies the **data type** of the value that the function returns
  - In the executable part of a function, **every** execution **path must** lead to a "*RETURN*" statement

```
DECLARE -- Declare and define function
    FUNCTION square (original NUMBER)   -- parameter list
    RETURN NUMBER                       -- RETURN clause
AS -- Declarative part begins
    original_squared NUMBER;
BEGIN                                   -- Executable part begins
    original_squared := original * original;
    RETURN original_squared;            -- RETURN statement
END;

BEGIN
  DBMS_OUTPUT.PUT_LINE(square(100));   -- invocation
END;
```

Back To Subprograms

Parameters

- If you **want** a subprogram to have **parameters**, **declare** formal **parameters** in the subprogram **heading**.

- When **invoking** the subprogram, specify the **actual parameters**

- **Data types** must be the **same** for the actual/formal parameters

Parameter Passing Methods

- By **reference**
- By **value**

By Reference

- If the subprogram **exits successfully**: the **value** of the **actual** parameter is the **final** value **assigned** to the **formal** parameter

- If the subprogram ends with an **exception**, then the value of the actual parameter is **undefined**

- "*OUT*" and "*IN OUT*" parameters can be returned in any order

```
CREATE OR REPLACE PACKAGE r_types AUTHID DEFINER IS
  TYPE r_type_1 IS RECORD (f VARCHAR2(5) := 'abcde');
  TYPE r_type_2 IS RECORD (f VARCHAR2(5));
END;
/

CREATE OR REPLACE PROCEDURE p (
  x IN r_types.r_type_1,
  y OUT r_types.r_type_2,
  z IN OUT VARCHAR2)
AUTHID CURRENT_USER IS
BEGIN
  DBMS_OUTPUT.PUT_LINE('x.f is ' || NVL(x.f,'NULL'));
  DBMS_OUTPUT.PUT_LINE('y.f is ' || NVL(y.f,'NULL'));
  DBMS_OUTPUT.PUT_LINE('z is ' || NVL(z,'NULL'));
END;
/
DECLARE
  r1 r_types.r_type_1;
  r2 r_types.r_type_2;
  s  VARCHAR2(5) := 'fghij';
BEGIN
  p (r1, r2, s);
END;
```

Parameter Aliasing

- Aliasing is having two **different names** for the **same** memory **location**

- Subprogram **parameter aliasing** always occurs when the compiler passes an **actual** parameter by **reference**, and can also occur when a subprogram has **cursor** variable parameters

```
CREATE OR REPLACE PACKAGE r_types AUTHID DEFINER IS
  TYPE r_type_1 IS RECORD (f VARCHAR2(5) := 'abcde');
  TYPE r_type_2 IS RECORD (f VARCHAR2(5));
END;
/

CREATE OR REPLACE PROCEDURE p (
  x IN r_types.r_type_1,
  y OUT r_types.r_type_2,
  z IN OUT VARCHAR2)
AUTHID CURRENT_USER IS
BEGIN
  DBMS_OUTPUT.PUT_LINE('x.f is ' || NVL(x.f,'NULL'));
  DBMS_OUTPUT.PUT_LINE('y.f is ' || NVL(y.f,'NULL'));
  DBMS_OUTPUT.PUT_LINE('z is ' || NVL(z,'NULL'));
END;
/
DECLARE
  r1 r_types.r_type_1;
  r2 r_types.r_type_2;
  s  VARCHAR2(5) := 'fghij';
BEGIN
  p (r1, r2, s);
END;
```

Forward Declaration

- If **nested** subprograms in the **same** PL/SQL **block invoke each other**, then **one requires** a **forward declaration**, because a subprogram **must be declared before** it can be invoked.

- A forward declaration declares a nested subprogram but does not define it. You must define it later in the same block.

```
DECLARE -- Declare proc1 (forward declaration):
PROCEDURE proc1(
    number1 NUMBER); -- Declare and define proc2:
  PROCEDURE proc2(
      number2 NUMBER)
  IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE ('Proc 2 Called With ' || number2);
    IF (number2 > 5) THEN
      DBMS_OUTPUT.PUT_LINE ('Proc 2 Is Done');
      proc1(number2 - 1);
    ELSE
      proc1(number2 + 1);
    END IF;
  END;
-- Define proc 1:
  PROCEDURE proc1(
      number1 NUMBER)
  IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE ('Proc 1 Called With ' || number1);
    IF (number1 > 7) THEN
      DBMS_OUTPUT.PUT_LINE ('Proc 1 Is Done');
    ELSE
      proc2(number1 + 2);
    END IF;
  END;
BEGIN
  proc1(0);
END;
```

Default Values

- When you declare a formal "*IN*" parameter, you can specify a **default** value for it
- A formal parameter with **no default** value is called a **required** parameter
- Omitting an actual parameter does not make the value of the corresponding formal parameter NULL

```
DECLARE
  first_name VARCHAR2(25);
  last_name  VARCHAR2(25);
  email      VARCHAR2(25);
  -- Declare and define procedure
PROCEDURE create_email( -- Subprogram heading begins
    name1   VARCHAR2,
    name2   VARCHAR2,
    company VARCHAR2 := 'stud.acs.upb.ro') -- Subprogram heading ends
IS                        -- Declarative part begins
  error_message VARCHAR2(30) := 'Email address is too long.';
BEGIN -- Executable part begins
  email := name1 || '.' || name2 || '@' || company;
EXCEPTION -- Exception-handling part begins
WHEN VALUE_ERROR THEN
  DBMS_OUTPUT.PUT_LINE(error_message);
END create_email;
BEGIN
  first_name := 'John';
  last_name  := 'Doe';
  create_email(first_name, last_name); -- invocation
  DBMS_OUTPUT.PUT_LINE ('With first name first, email is: ' || email);
  create_email(last_name, first_name, 'cs.pub.ro'); -- invocation
  DBMS_OUTPUT.PUT_LINE ('With last name first, email is: ' || email);
  first_name := 'Elizabeth';
  last_name  := 'MacDonald';
  create_email(first_name, last_name, 'cs.pub.ro'); -- invocation
END;
```

Positional, Named, and Mixed Notation

- Positional: Specify the **actual** parameters in the **same order** as the **formal** parameters are declared
- Named: Specify the actual parameters in **any order**, using this **syntax** = “formal => actual”
- Mixed: Start with positional notation, then use named notation for the remaining parameters

Positional

- Specifying actual parameters in the **wrong order** can cause **problems** that are hard to detect, especially if the actual parameters are literals
- Subprogram invocations **must change if** the formal parameter **list changes**, unless the list only acquires new trailing optional parameters
- **Reduced** code **clarity** and **maintainability**. **Not recommended** if the subprogram has a **large number of parameters**

Named

- There is **no wrong order** for specifying actual parameters
- Subprogram invocations **must change only** if the formal parameter list acquires **new required parameters**
- **Recommended** when you invoke a subprogram defined or maintained by **someone else**

Mixed

- Since you start with positional you get all disadvantages from them, but it can be solved by using the named parameters


CALLING
YOUR PARAMETERS
BY NAME
1, 23,
'SOMETHING',
'2019-10-16', 3
imgflip.com

```
DECLARE
  emp_num NUMBER(6) := 120;
  bonus   NUMBER(6) := 50;

  PROCEDURE raise_salary (
    emp_id NUMBER,
    amount NUMBER
  ) IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE ('Called for ' || emp_id || ' with the bonus of ' || amount);
  END raise_salary;

BEGIN
  -- Equivalent invocations:

  raise_salary(emp_num, bonus);                       -- positional notation
  raise_salary(amount => bonus, emp_id => emp_num);   -- named notation
  raise_salary(emp_id => emp_num, amount => bonus);   -- named notation
  raise_salary(emp_num, amount => bonus);             -- mixed notation
END;
```

Cursor Variable Parameters

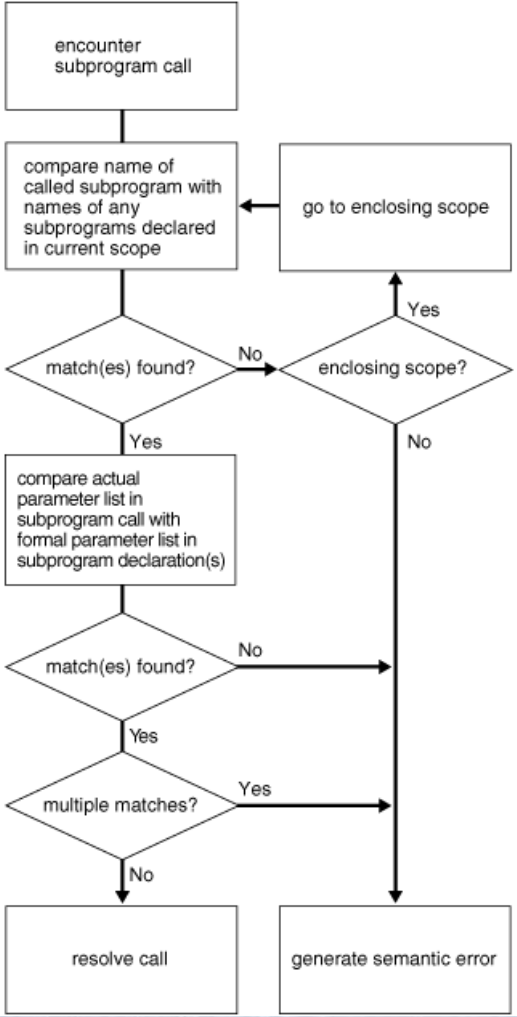
Cursor variable parameters **are pointers**. Therefore, if a subprogram **assigns one** cursor variable parameter to **another**, they refer to the **same memory location**. This aliasing can have **unintended results**.

```
DECLARE
  TYPE EmpCurTyp IS REF CURSOR;
  c1 EmpCurTyp;
  c2 EmpCurTyp;

  PROCEDURE get_emp_data (
    emp_cv1 IN OUT EmpCurTyp,
    emp_cv2 IN OUT EmpCurTyp
  )
  IS
    emp_rec emp%ROWTYPE;
  BEGIN
    OPEN emp_cv1 FOR SELECT * FROM emp;
    emp_cv2 := emp_cv1;  -- now both variables refer to same location
    FETCH emp_cv1 INTO emp_rec;  -- fetches first row of employees
    FETCH emp_cv1 INTO emp_rec;  -- fetches second row of employees
    FETCH emp_cv2 INTO emp_rec;  -- fetches third row of employees
    CLOSE emp_cv1;  -- closes both variables
    FETCH emp_cv2 INTO emp_rec; -- causes error when get_emp_data is invoked
  END;
BEGIN
  get_emp_data(c1, c2);
END;
```

Subprogram Invocation Resolution

- When the PL/SQL **compiler** encounters a subprogram **invocation**, it searches for a **matching** subprogram **declaration—first** in the **current scope** and then, if necessary, in successive **enclosing scopes**
- A **declaration** and **invocation match** if their subprogram **names** and **parameter** lists **match**
- If the compiler finds no matching declaration for an invocation, then it generates a semantic error


encounter subprogram call
compare name of called subprogram with names of any subprograms declared in current scope
go to enclosing scope
match(es) found?
No
enclosing scope?
Yes
Yes
No
compare actual parameter list in subprogram call with formal parameter list in subprogram declaration(s)
match(es) found?
No
Yes
multiple matches?
Yes
No
resolve call
generate semantic error

Overloaded Subprograms

- PL/SQL lets you **overload nested** subprograms, **package subprograms**, and **type methods**

- You can use the **same name** for several **different subprograms** if their formal parameters **differ in name**, **number**, **order**, or **data type** family

```
 1 DECLARE
 2   PROCEDURE initialize (tab IN VARCHAR, n INTEGER) IS
 3   BEGIN
 4     DBMS_OUTPUT.PUT_LINE('Invoked first version');
 5   END initialize;
 6
 7   PROCEDURE initialize (tab IN INT, n INTEGER) IS
 8   BEGIN
 9     DBMS_OUTPUT.PUT_LINE('Invoked second version');
10   END initialize;
11
12 BEGIN
13   initialize('Prima', 50);
14   initialize(2, 100);
15 END;
```

You can Overload

- You **can overload** subprograms if their formal parameters **differ only** in **numeric** data type.

- A **VARCHAR2** value **can** match a **NUMBER**, **BINARY_FLOAT**, or **BINARY_DOUBLE** parameter.

You canNOT Overload

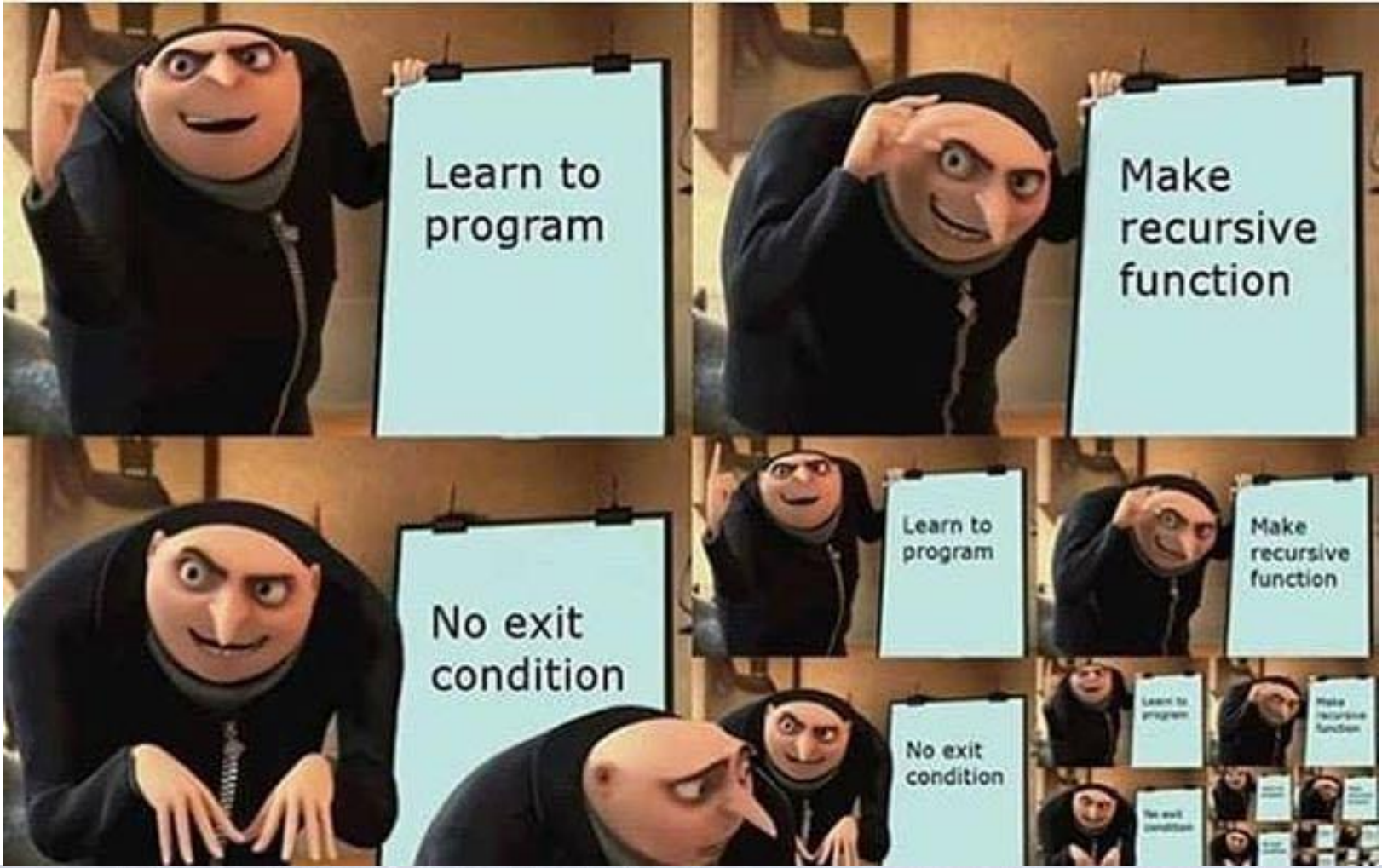
- **Standalone** subprograms
- Subprograms whose formal parameters **differ** only in **mode**
- Subprograms whose formal parameters **differ** only in **subtype**
- **Functions** that **differ** only in **return value** data type, even if the data types are in different families

Recursive Subprograms

- A **recursive** subprogram **invokes itself**. Recursion is a powerful technique for simplifying an algorithm

- A recursive subprogram **must have at least two execution paths—one** leading to the **recursive invocation** and **one** leading to a **terminating condition**

```
1 CREATE OR REPLACE FUNCTION factorial (
2   n POSITIVE
3 ) RETURN POSITIVE
4   AUTHID DEFINER
5 IS
6 BEGIN
7   IF n = 1 THEN                    -- terminating condition
8     RETURN n;
9   ELSE
10    RETURN n * factorial(n-1);   -- recursive invocation
11  END IF;
12 END;
13 /
14 BEGIN
15   FOR i IN 1..5 LOOP
16     DBMS_OUTPUT.PUT_LINE(i || '! = ' || factorial(i));
17   END LOOP;
18 END;
```


Learn to program
Make recursive function
No exit condition
Learn to program
Make recursive function
No exit condition

Side Effects

- A subprogram has **side effects** if it changes **anything except** the values of its **own local variables**. For example affecting:
  - Its own **OUT** or **IN OUT** parameter
  - A **global** variable
  - A **public** variable in a package
  - A database **table**
  - The **database**
  - The external **state** (by invoking DBMS_OUTPUT or sending e-mail, for example)

Back To Functions

Function Result Cache

When a PL/SQL function has the "*RESULT_CACHE*" option, its results are cached in the shared global area (**SGA**) so sessions connected to the same instance can reuse these results when available.

Functions that SQL Statements Can Invoke

- When invoked from a "*SELECT*" statement or a parallelized "*INSERT*", "*UPDATE*", "*DELETE*", or "*MERGE*" statement, the subprogram **cannot modify** any database tables
- When invoked from an "*INSERT*", "*UPDATE*", "*DELETE*", or "*MERGE*" statement, the subprogram **cannot query or modify** any database tables **modified by that statement**

Functions that SQL Statements Can Invoke

- When invoked from a "*SELECT*", "*INSERT*", "*UPDATE*", "*DELETE*", or "*MERGE*" statement, the subprogram **cannot execute any** of the following SQL statements (unless PRAGMA AUTONOMOUS_TRANSACTION was specified):
  - **Transaction control statements** (such as COMMIT)
  - **Session control statements** (such as SET ROLE)
  - **System control statements** (such as ALTER SYSTEM)
  - Database definition language (**DDL**) statements (such as CREATE), which are committed automatically

Back To Subprograms

External Subprograms

- If a **C procedure** or **Java method** is **stored** in the database, you can **publish** it **as an external** subprogram

- To **publish** an external subprogram, **define** a stored PL/SQL **subprogram** with a **call specification**

- **Invoke** the published external subprogram by its PL/SQL name

```
1 import java.sql.*;
2 import oracle.jdbc.driver.*;
3 public class Adjuster {
4   public static void raiseSalary (int empNo, float percent)
5   throws SQLException {
6     Connection conn = new OracleDriver().defaultConnection();
7     String sql = "UPDATE employees SET salary = salary * ?
8                   WHERE employee_id = ?";
9     try {
10      PreparedStatement pstmt = conn.prepareStatement(sql);
11      pstmt.setFloat(1, (1 + percent / 100));
12      pstmt.setInt(2, empNo);
13      pstmt.executeUpdate();
14      pstmt.close();
15    } catch (SQLException e)
16        {System.err.println(e.getMessage());}
17    }
18 }
```

```
CREATE OR REPLACE PROCEDURE raise_salary (
  empid NUMBER,
  pct   NUMBER
) AS
  LANGUAGE JAVA NAME 'Adjuster.raiseSalary (int, float)';  -- call specification
/

BEGIN
  raise_salary(120, 10);  -- invoke Adjuster.raiseSalary by PL/SQL name
END;
```


That's all Folks!